

Curs 1

1. Timpul de răspuns : timpul necesar terminării unui task

-Include accesele la memorie, operațiile de I/E și operațiile executate de sistemul de operare

2. Timpul UCP: timpul în care UCP execută efectiv un program

-Nu cuprinde timpul de așteptare pentru operațiile de I/E

-Nu include nici timpul în care UCP execută alte programe

-Poate fi divizat în: Timpul UCP al utilizatorului, Timpul UCP al sistemului

3. Exprimarea timpului UCP:

- Timpul UCP (t_{UCP}) poate fi exprimat prin:

$$t_{UCP} = C_{UCP} \times t_C$$

C_{UCP} – numărul ciclurilor de ceas ale UCP necesare pentru execuția programului

t_C – durata ciclului de ceas

- O altă exprimare:

$$t_{UCP} = \frac{C_{UCP}}{f}$$

f – frecvența semnalului de ceas

4. Indicatorul MIPS:

-Cel mai important indicator de performanță: timpul de execuție al programelor reale

-Totuși, s-au adoptat diferiți indicatori populari de performanță

-Unul din indicatori este numit MIPS (Millions of Instructions Per Second)

-Indică numărul de “instrucțiuni medii” pe care un calculator le poate executa pe secundă

-Pentru un program dat, MIPS este:

$$\text{MIPS} = \frac{N}{t_E \times 10^6}$$

N – contorul de instrucțiuni

Considerând că $t_E = t_{UCP}$,

$$t_E = \frac{N \times \text{CPI}}{f}$$

Rezultă:

$$\text{MIPS} = \frac{f}{\text{CPI} \times 10^6}$$

- Timpul de execuție exprimat în funcție de indicatorul MIPS:

$$t_E = \frac{N}{\text{MIPS} \times 10^6}$$

-Un indicator similar: BIPS (Billions of Instructions Per Second) sau GIPS

-Avantajul indicatorului MIPS: este ușor de înțeles, mai ales de către utilizatori

5. Probleme legate de utilizarea indicatorului MIPS:

-Există anumite probleme atunci când MIPS este utilizat ca o măsură pentru comparație:

- MIPS este dependent de setul de instrucțiuni
- MIPS variază pentru programe diferite ale aceluiași calculator
- MIPS poate varia invers proporțional cu performanța

6. Indicatorul MFLOPS (Millions of Floating-point Operations Per Second)

- Formula de calcul:

$$\text{MFLOPS} = \frac{N_{VM}}{t_E \times 10^6}$$

NVM – numărul de operații în virgulă mobilă

-Valoarea MFLOPS este dependentă de calculator și de program

7. Probleme legate de utilizarea indicatorului MFLOPS:

- Setul operațiilor de calcul în VM diferă de la un calculator la altul

- Valoarea MFLOPS se modifică în funcție de:

- Combinația operațiilor întregi și în VM
- Combinația operațiilor în VM mai rapide și mai lente

-Soluția la ambele probleme: utilizarea operațiilor normalizate în VM

Curs 2

1. Metrice pentru sintetizarea performanței unui grup de programe de evaluare:

-Modul în care se poate sintetiza performanța unui grup de programe de evaluare :

-Timpul total de execuție : Metoda cea mai simplă

-Media aritmetică a timpilor de execuție :

$$MA = \frac{1}{n} \sum_{i=1}^n t_{Ei}$$

t_{Ei} – timpul de execuție al programului i din totalul de n programe din set

-Media aritmetică ponderată a timpilor de execuție :

- Utilizare: dacă numărul de execuții ale programelor dintr-un set este diferit
- Se asignează fiecărui program o pondere p_i → indică frecvența sa de execuție
- Alegerea ponderilor: pe un calculator de referință timpii de execuție ponderați ai fiecărui program să fie egali

-Normalizarea timpilor de execuție față de un calculator de referință :

- Se consideră media timpilor de execuție normalizați
- Dacă se utilizează media aritmetică a timpilor de execuție: rezultatul depinde de alegerea calculatorului de referință

-Media geometrică a timpilor de execuție :

$$MG = \sqrt[n]{\prod_{i=1}^n t_{Ei}}$$

- Este independentă de seria datelor utilizate pentru normalizare
- Are proprietatea:

$$\frac{MG(X_i)}{MG(Y_i)} = MG\left(\frac{X_i}{Y_i}\right)$$

- Rezultatul este același indiferent de calculatorul de referință
- Avantajele mediei geometrice:
 - Este independentă de timpii de execuție ai programelor individuale
 - Nu are importanță care calculator este utilizat pentru normalizare
- Dezavantajul mediei geometrice: Nu anticipează performanțele

2. Programe artificiale (sintetice) de evaluare a performanțelor :

-Programe artificiale sau sintetice de evaluare :

- Scopul: crearea unor programe în care frecvențele de execuție ale instrucțiunilor sunt aceleași cu cele dintr-un set de programe de evaluare

- Programul sintetic Whetstone :

- Publicat de National Physical Laboratory (NPL), Marea Britanie
- Bazat pe măsurătorile efectuate asupra aplicațiilor științifice și ingineresti scrise în limbajul Algol 60
- Numit după compilatorul Whetstone Algol de la NPL
- Rescris ulterior în limbajele Fortran și Pascal
- Pune accent pe operațiile în VM

-Programul sintetic Dhystone :

- Creat pentru evaluarea programelor de sistem
- Bazat pe un set de măsurători ale frecvențelor de execuție ale instrucțiunilor

- Nu pune accent pe operațiile în VM Scris inițial în limbajul Ada
- Convertit ulterior în limbajul C
- Dezavantajele programelor sintetice :
 - Nu reflectă comportamentul programelor reale
 - Optimizările executate de compilator sau prin hardware pot amplifica performanțele acestor programe

3. Programe nucleu (kernel) de evaluare a performanțelor :

- Fragmente de dimensiuni mici, dar solicitante, extrase din programe reale –
- Elaborate pentru evaluarea calculatoarelor performante (supercalculatoare)
- Exemple: Livermore Loops și Linpack
- Utilizate în special pentru evaluarea performanțelor aplicațiilor științifice

4. Setul de programe de evaluare SPEC(Standard Performance Evaluation Corporation) CPU2017

- A fost dezvoltat de grupul OSG
- Se măsoară performanța UCP, a sistemului de memorie și a generării codului de către compilator
- Timpul necesar executării funcțiilor SO și a operațiilor de I/E este neglijabil
- Conține porțiuni din 43 de aplicații reale în limbajele C, C++ și FORTRAN
- Aplicații portabile pe arhitecturi variate: IA32; AMD64 (x86-64); ARMv8 AArch64; SPARC etc.
- Îmbunătățiri față de versiunea precedentă:
 - Complexitate și dimensiune crescută a programelor de evaluare
 - Posibilitatea utilizării interfeței de programare OpenMP (Open Multi-Processing) pentru evaluarea sistemelor paralele
 - Metrică opțională pentru măsurarea consumului de putere
 - Se poate măsura puterea maximă, puterea medie și energia totală consumată (în kJ)
 - Două tipuri de măsurători
 - Măsurarea vitezei de execuție:
 - Viteza SPEC: timpul în care se execută toate programele din setul de test o singură dată SPECspeed 2017 Integer;
 - SPECspeed 2017 Floating Point
 - Măsurarea ratei:

-Rata SPEC: numărul de programe care pot fi executate într-o unitate de timp (pe oră)

-SPECrate 2017 Integer; SPECrate 2017 Floating Point

- Se calculează media geometrică a indicatorilor

- Timpul de execuție sunt raportați la un calculator de referință :

-Sun Microsystems Sun Fire V490, cu procesoare UltraSPARC-IV+ (2,1 GHz)

-Programele au fost rulate pe acest calculator pentru a stabili un timp de

referință

-Pentru calculatorul de referință, metricile SPECspeed2017 și SPECrate2017

sunt 1

-Exemple de programe pentru numere întregi :

- 502.gcc_r: compilator C bazat pe GCC vers. 4.5.0

-520.omnetpp_r: simulare discretă a evenimentelor dintr-o rețea Ethernet de 10 Gbiți/s pe baza simulatorului OMNeT++

-525.x264_r: compresie video în formatul H.264/MPEG-4 AVC

-531.deepsjeng_r: bazat pe programul de șah Deep Sjeng

-548.exchange2_r: generator pentru jocul sudoku

-Exemple de programe pentru numere în VM :

-508.namd_r: simularea unor sisteme biomoleculare

-511.povray_r: trasarea razelor pe baza programului POV-Ray (vers. 3.7)

-521.wrf_r: predicția vremii pe baza modelului WRF, vers. 3.6.1

-526.blender_r: redare 3D, animație (fundația Blender)

-538.imagick_r: prelucrări de imagini

5. Legea lui Amdahl indică creșterea vitezei care se poate obține dacă se îmbunătățește o parte a unui calculator

-Factori de care depinde creșterea vitezei: fracțiunea de timp în care se poate utiliza îmbunătățirea; creșterea vitezei obținute dacă îmbunătățirea s-ar utiliza permanent

-Poate indica modul în care trebuie distribuite resursele pentru a îmbunătăți raportul cost/performanță

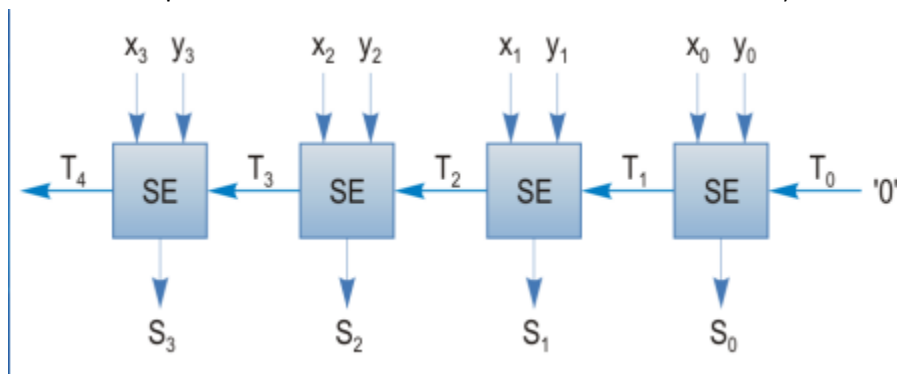
Curs 3

1. Sumator cu propagare succesivă a transportului ("Ripple Carry Adder")

- Algoritmul de adunare cu propagarea succesivă a transportului

$$\begin{array}{r} x_3 x_2 x_1 x_0 + \\ y_3 y_2 y_1 y_0 \\ \hline s_4 s_3 s_2 s_1 s_0 \end{array}$$

- O posibilitate de implementare: conectarea mai multor sumatoare elementare în serie, câte un sumator elementar pentru fiecare bit
- Schema bloc pentru adunarea a două numere binare de câte 4 biți



- Este un sumator paralel
- Transportul trebuie să se propage succesiv prin toate sumatoarele înainte de a se cunoaște rezultatul final
- Avantaje: simplitate, cost redus
- Dezavantaj: viteză redusă
- Poate fi utilizat și ca scăzător

• Presupunem numerele binare:

$$X = 0\ 1001\ (+9)$$
$$Y = 0\ 0011\ (+3)$$

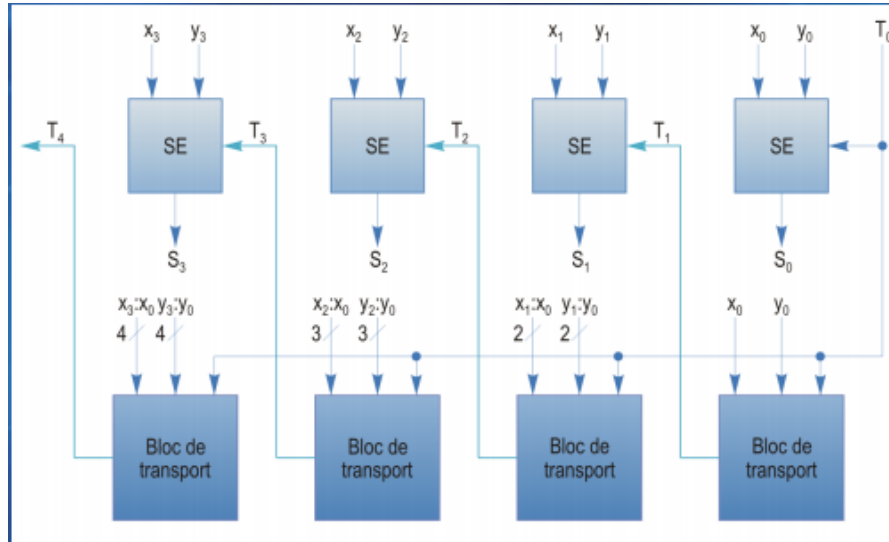
• Scăderea $X - Y$ poate fi executată astfel:

X	0	1	0	0	1	+9
C2(Y)	1	1	1	0	1	-3
X-Y	⊕	0	0	1	1	+6

• Transportul de la poziția c.m.s. se neglijează

2. Sumator cu anticiparea transportului (“Carry Lookahead Adder”)

- Reduce timpul necesar pentru formarea semnalelor de transport
- Intrarea de transport necesară pentru un etaj este generată în mod direct
- Schema bloc a unui sumator cu anticiparea transportului de 4 biți →



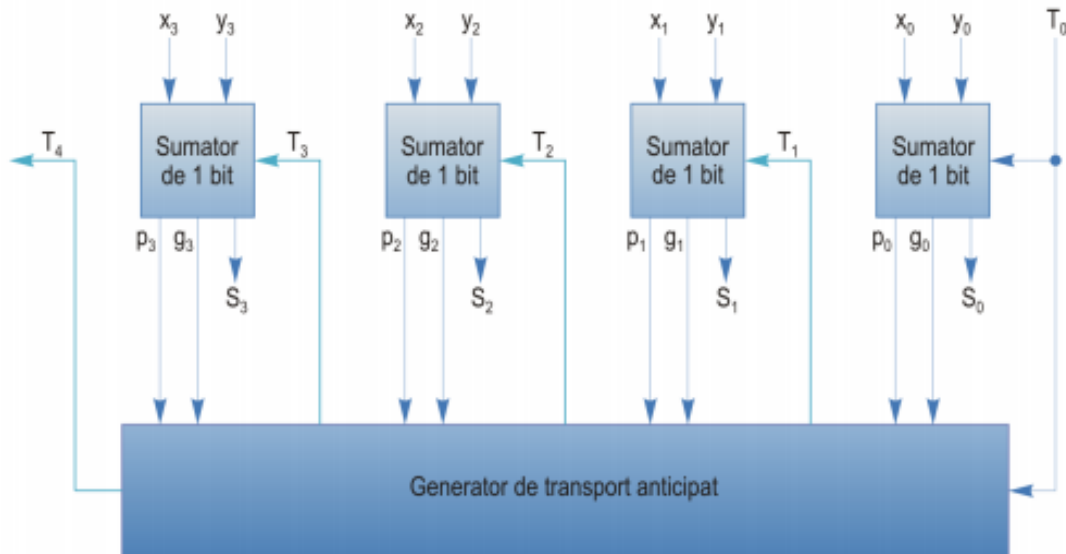
-
- Transportul de ieșire al unui sumator elementar: $T_{i+1} = x_i y_i + (x_i + y_i) T_i$
- Expresiile pentru T_1 și T_2 :
 - $T_1 = x_0 y_0 + (x_0 + y_0) T_0$
 - $T_2 = x_1 y_1 + (x_1 + y_1) T_1 = x_1 y_1 + (x_1 + y_1) [x_0 y_0 + (x_0 + y_0) T_0]$

3. Funcțiile p și g pentru propagarea și generarea transportului pe bit

- Funcția g → generarea transportului $g_i = x_i y_i$
- Funcția p → propagarea intrării de transport la ieșirea de transport $p_i = x_i + y_i$
- Transportul de ieșire: $T_{i+1} = g_i + p_i T_i$

4. Funcțiile P și G pentru un grup de biți

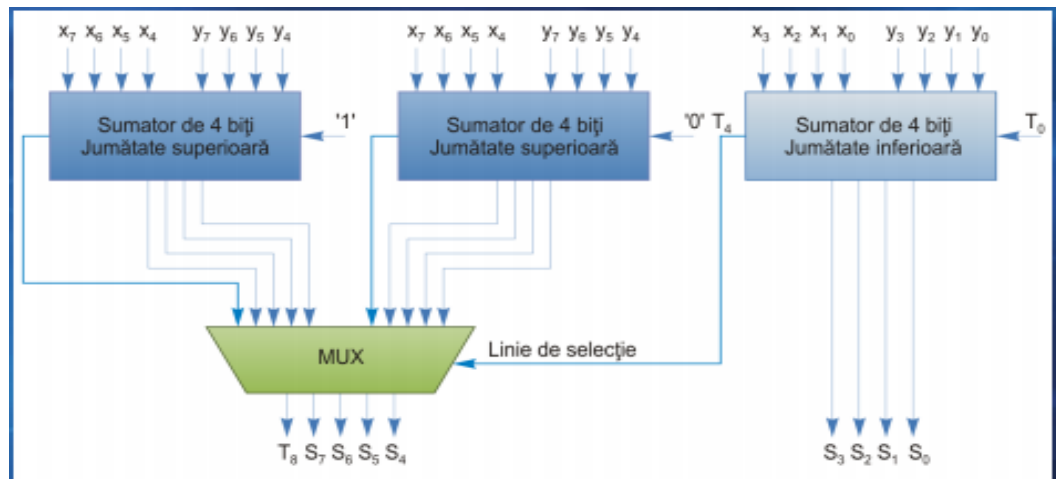
- Schema bloc modificată a sumatorului cu anticiparea transportului de 4 biți →



- Pentru un grup de 4 biți:
- $G_{0,3} = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$
- $P_{0,3} = p_3 p_2 p_1 p_0$
- Rezultă: $T_4 = G_{0,3} + P_{0,3} T_0$
-

5. Principiul sumatorului cu selecția transportului ("Carry Select Adder")

- Utilizează circuite redundante pentru creșterea vitezei de adunare
- Se calculează jumătatea superioară a sumei pentru ambele valori posibile ale transportului
- Atunci când transportul este cunoscut, este selectată jumătatea superioară corectă a sumei
- Exemplu pentru numere de câte 8 biți →



6. Principiul sumatorului cu salvarea transportului (“Carry Save Adder”)

- Utilizat atunci când trebuie adunate mai mult de două numere
- Reduce timpul de propagare al semnalelor de transport
- SST de n biți: colecție de n sumatoare elementare independente
 - Intrări: trei numere de câte n biți
 - Ieșiri: un cuvânt sumă S de n biți, un cuvânt de transport T de n biți
- Fiecare sumator elementar funcționează independent unul de celălalt
- Semnalele de transport nu sunt propagate între sumatoarele elementare
- Pentru obținerea rezultatului final, suma și transportul trebuie adunate utilizând un sumator obișnuit → sumator cu propagarea transportului (SPT)

Prima etapă

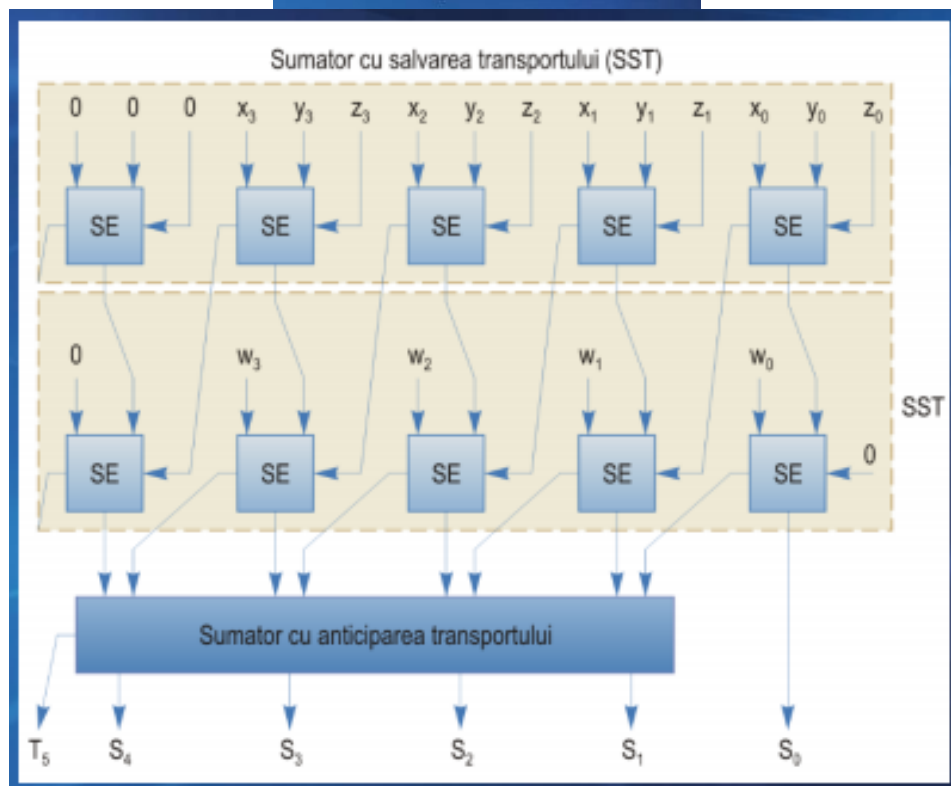
X	0101
Y	0011
Z	<u>0100</u>
Suma	0010
Transportul salvat	1010

Etapa a doua

W	0001
Suma	0010
Transportul salvat	<u>1010</u>
Noua sumă	1001
Noul transport salvat	0100

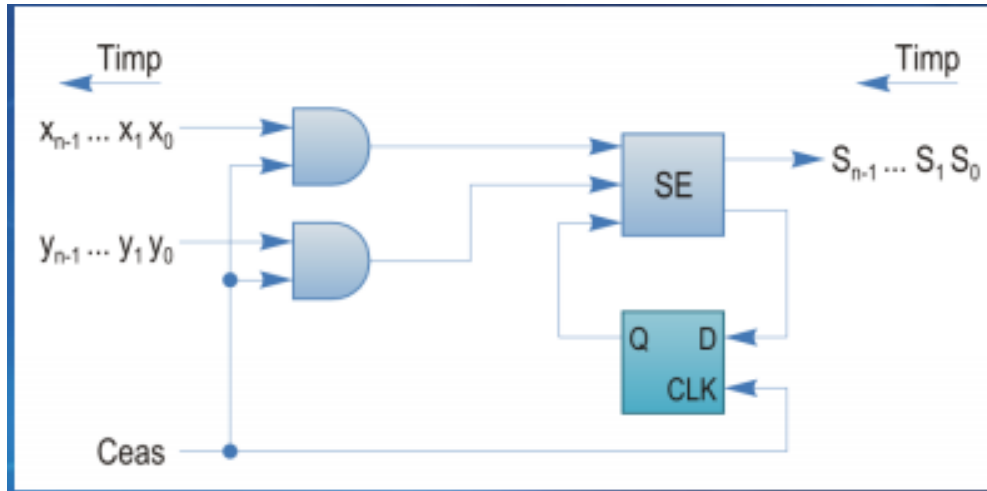
Etapa a treia

Noua sumă	1001
Noul transport salvat	<u>0100</u>
Rezultat	1101 (13)



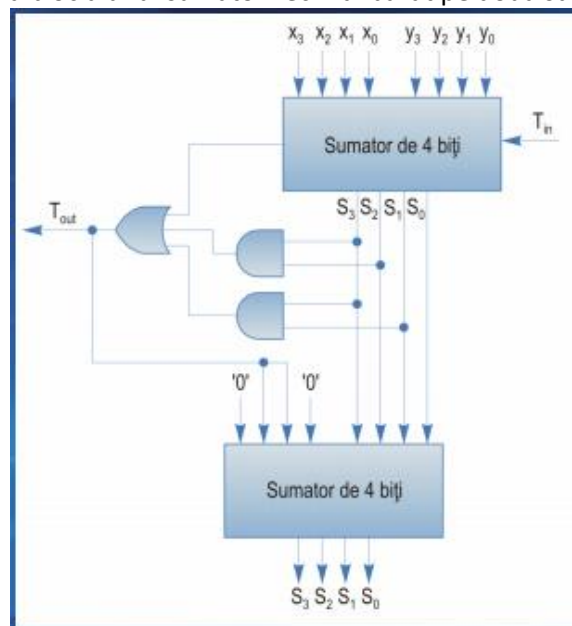
7. Sumator serial

- Execută adunarea pas cu pas începând cu bitul c.m.p.s.
- Un singur sumator elementar și un bistabil D (latch) utilizat pentru propagarea transportului sumei biților de ordin i la suma biților de ordin $i+1$
- Avantaje: simplitate, cost redus



8. Sumator zecimal

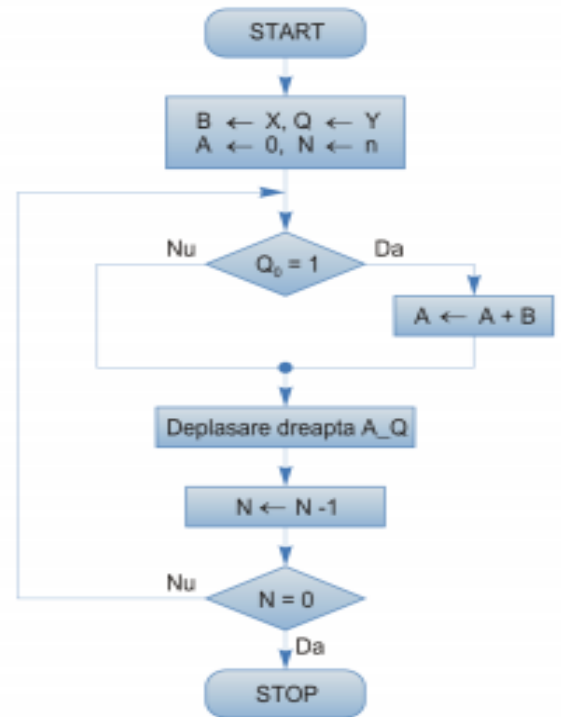
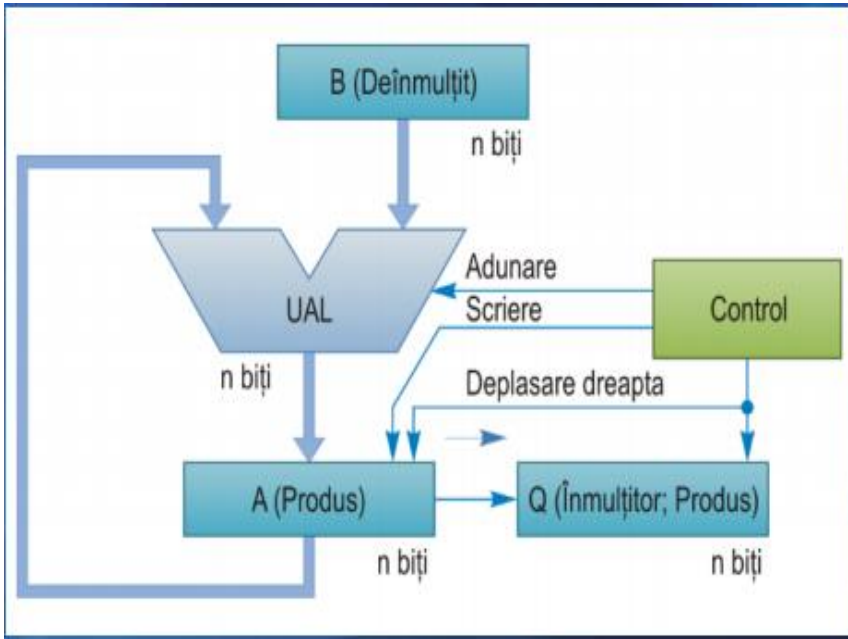
- Utilizat pentru numere reprezentate în zecimal (BCD)
- Adună două cifre BCD în paralel
- Generează o sumă în cod BCD
- Dacă suma depășește valoarea 9 sau se generează un transport către cifra următoare, rezultatul trebuie corectat:
 - Corecția: se adună valoarea 6 la rezultat
- Schema bloc a unui sumator zecimal bazat pe două sumatoare de 4 biți →



Curs 4

1. Versiunea finală a circuitului + algoritmul de înmulțire prin deplasare și adunare

- Versiunea finală a circuitului de înmulțire combină produsul (registrul A) cu înmulțitorul (registrul Q)
 - Registrul A este de numai n biți
 - Produsul este format în registrele A și Q



2. Execuția operației de înmulțire prin tehnica Booth

- Tehnica Booth : Ideea principală: dacă se poate efectua atât adunare, cât și scădere, există mai multe posibilități de a calcula un produs
- La înmulțirea prin tehnica Booth se consideră câte doi biți adiacenți ai înmulțitorului pentru a determina operația care trebuie efectuată

Y_i	Y_{i-1}	Operații
0	0	Deplasare la dreapta
0	1	Adunare de înmulțit, deplasare la dreapta
1	0	Scădere de înmulțit, deplasare la dreapta
1	1	Deplasare la dreapta

Pas	A	Q (Y)	Q_1	Q_0Q_1	B (X)	Operații
0	0 0000	1 0110	0	0 0	0 1101	Inițializare
1	0 0000	0 1011	0	1 0	0 1101	shr (A_Q)
2	1 0011	0 1011	0		0 1101	$A \leftarrow A - B$
	<u>1</u> 1001	1 0101	1	1 1		shr (A_Q)
3	<u>1</u> 1100+	1 1010	1	0 1	0 1101	shr (A_Q)
4	<u>0</u> 1101				0 1101	$A \leftarrow A + B$
	0 1001	1 1010	1			
	0 0100+	1 1101	0	1 0		shr (A_Q)
5	<u>1</u> 0011				0 1101	$A \leftarrow A - B$
	1 0111	1 1101	0			
	<u>1</u> 1011	1 1110	1	0 1		shr (A_Q)

3. Avantaje ale tehnicii de înmulțire Booth

- Elimină necesitatea conversiei operandilor la forma pozitivă
- Poate reduce numărul operațiilor necesare

Curs 5 Inmultire 2

1. Principiul înmulțirii într-o bază superioară

- Înmulțirea într-o bază superioară reduce numărul etapelor prin examinarea mai multor biți ai înmulțitorului în fiecare etapă
 - Înmulțire în baza 4: sunt examinați 2 biți
 - Înmulțire în baza 8: sunt examinați 3 biți

Se examinează mai mulți biți ai înmulțitorului Y în fiecare pas → crește viteza operației

- Înmulțire în baza 4: sunt examinați 2 biți
 - 00: nu se execută adunare
 - 01: se adună X la produsul parțial
 - 10: se adună 2X la produsul parțial
 - 11: se adună X + 2X la produsul parțial
 - Calculul X + 2X poate fi evitat prin utilizarea tehnicii Booth în baza 4

Un alt nume al tehnicii Booth în baza 4: înmulțirea cu tripleți suprapuși

- Registrul A trebuie să aibă o poziție suplimentară

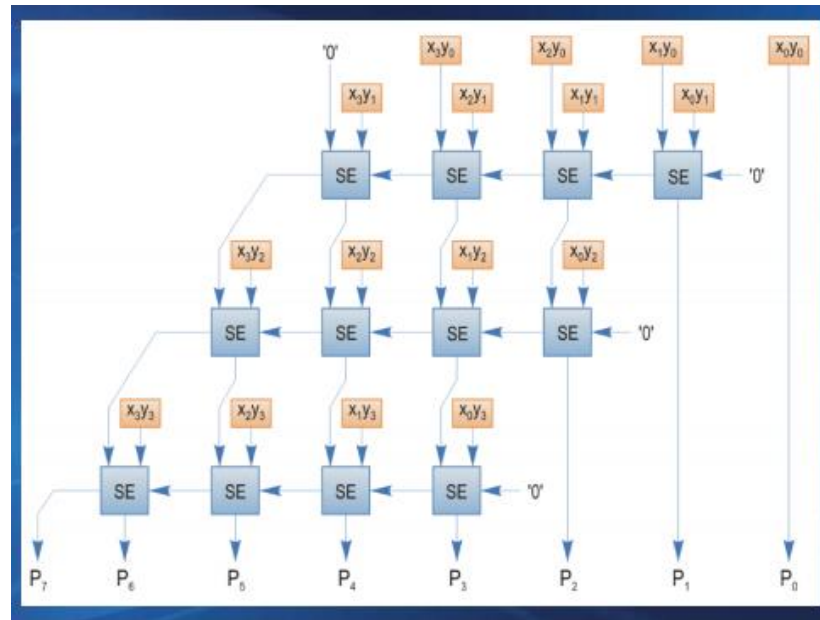
Avantaj suplimentar: metoda poate fi utilizată pentru numere fără semn și pentru numere cu semn reprezentate în C2

Versiunea în baza 8 a tehnicii Booth: trebuie să se genereze valoarea 3X

2. Tehnica de înmulțire Booth în baza 4

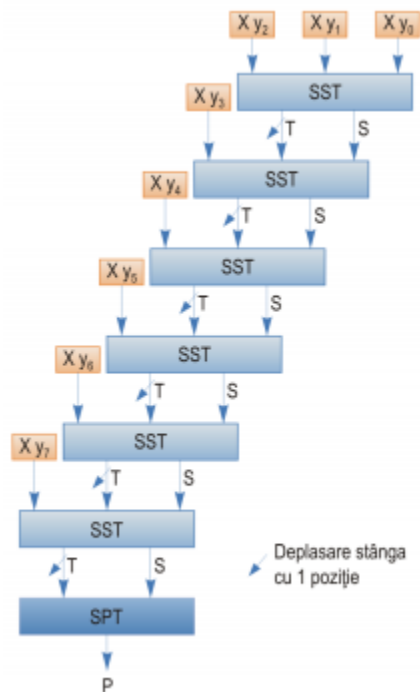
3. Principiul înmulțirii matriceale

- Înmulțirea matriceală utilizează o matrice de circuite combinaționale pentru creșterea vitezei operației
 - Se pot utiliza sumatoare cu salvarea transportului pentru amânarea propagării transportului până la ultimul etaj
 - O matrice de $n \times n$ porți Φ poate calcula toți termenii $x_i * y_j$ simultan
 - Termenii sunt însumați cu o matrice de $n(n-1)$ sumatoare elementare
 - Circuitul rezultat este similar cu un sumator bidimensional cu transport succesiv
 - Deplasările implicate de factorii 2ⁱ și 2^j sunt implementate prin deplasarea spațială a sumatoarelor pe direcția x și y

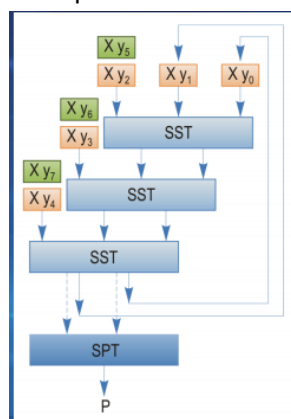


4. Circuit de înmulțire matriceală utilizând sumatoare cu propagarea succesivă a transportului

- Pentru creșterea vitezei se pot utiliza sumatoare cu salvarea transportului (SST)
 - Propagarea transportului între sumatoarele elementare din același rând este eliminată
 - Propagarea transportului este amânată până la ultimul etaj al circuitului
- Exemplu: Circuit de înmulțire matriceală pentru numere de câte 8 biți care utilizează SST



- Circuitul anterior este practic pentru valori moderate ale lui n
- Pentru valori mari ale lui n , este necesar un număr mare de sumatoare SST
- Sumatorul poate fi partiționat în k segmente de câte m biți fiecare
 - Sunt generate doar m produse parțiale
 - Procesul este repetat de k ori
- Exemplu: Structură cu două treceri



5. Circuit de înmulțire utilizând celula de 1 bit de înmulțire și adunare

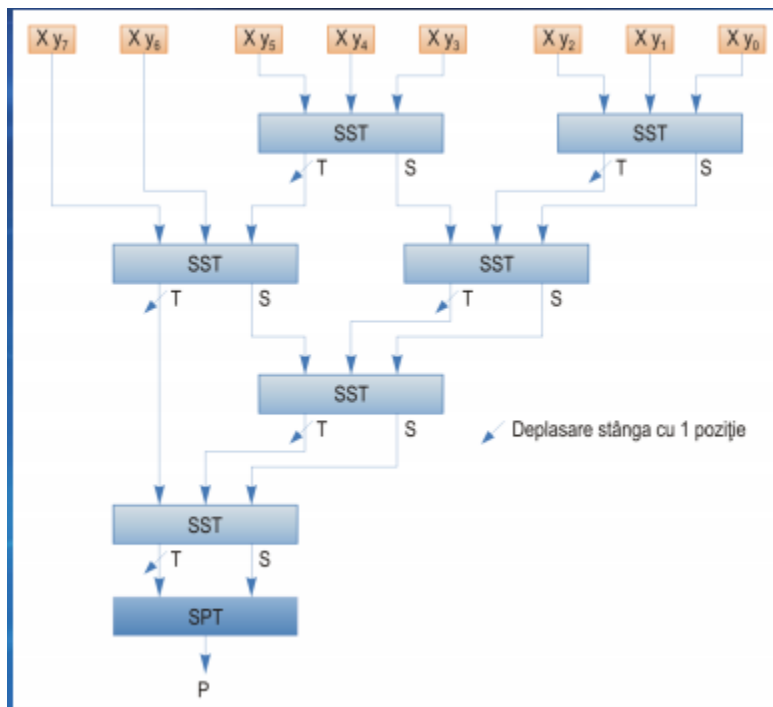
- Celula implementează expresia aritmetică: $Tout\ S = a\ plus\ x*y\ plus\ Tin$
- La intrarea a se conectează un bit al produsului parțial din linia precedentă
- Un circuit de înmulțire de $n \times n$ biți poate fi realizat utilizând n^2 celule de acest tip
 - Unele celule vor avea intrările setate la '0'
- Avantaj: structură uniformă → permite implementarea într-un circuit VLSI

6. Circuit de înmulțire matriceală utilizând sumatoare cu salvarea transportului

- Pentru creșterea vitezei se pot utiliza sumatoare cu salvarea transportului (SST)
 - Propagarea transportului între sumatoarele elementare din același rând este eliminată
 - Propagarea transportului este amânată până la ultimul etaj al circuitului
 - Exemplu: Circuit de înmulțire matriceală pentru numere de câte 8 biți care utilizează SST

7. Principiul arborelui Wallace

- Arborele Wallace se utilizează pentru adunarea mai rapidă a unor produse parțiale
 - Produsele parțiale sunt grupate câte trei
 - Se utilizează câte un SST pentru adunarea lor

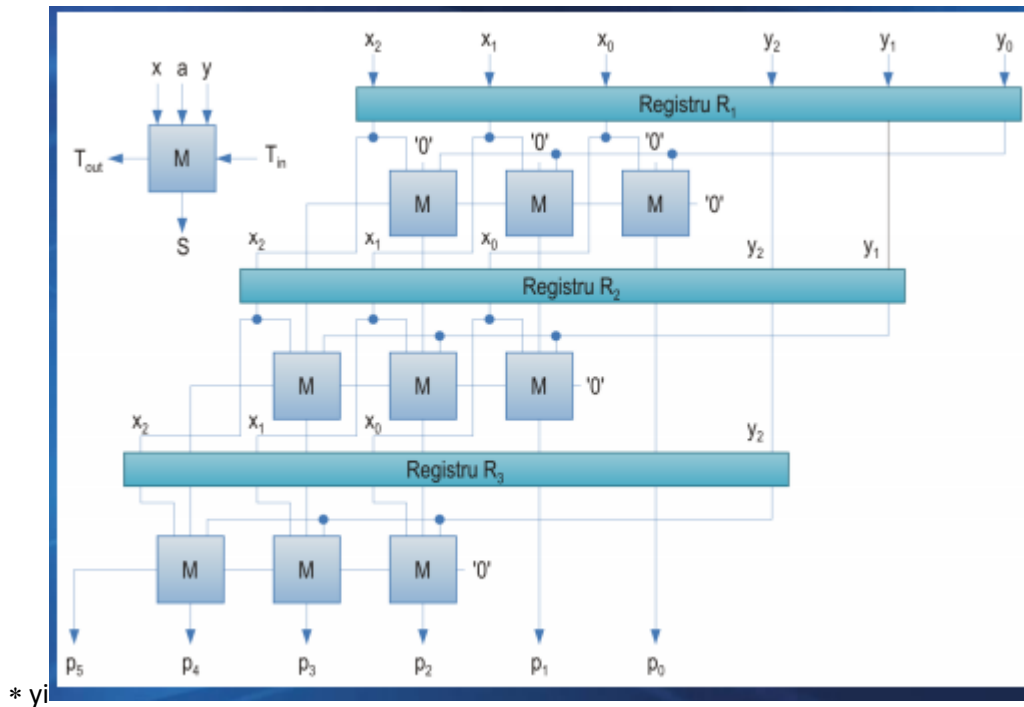


7. Circuit de înmulțire combinațională utilizând un arbore Wallace pentru însumarea produselor parțiale

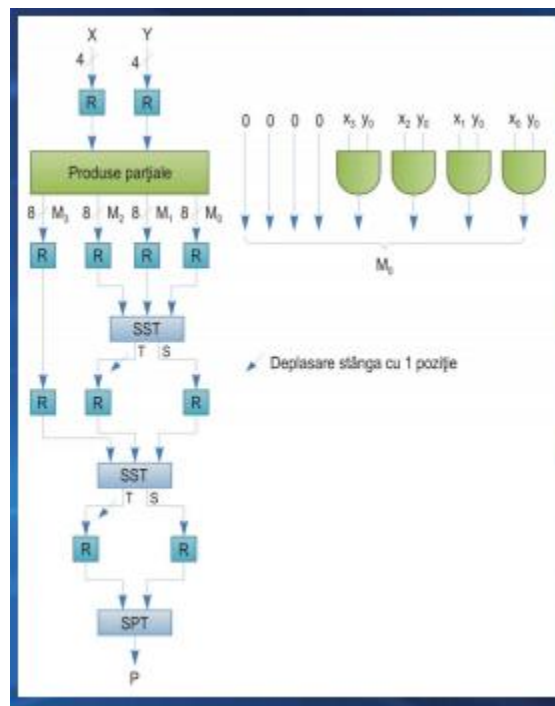
- Înmulțirea a două numere de câte n biți necesită adunarea a n produse parțiale
- Circuitele de înmulțire anterioare execută înmulțirea într-un timp $O(n)$
- Timpul poate fi redus la $O(\log n)$ prin utilizarea unui arbore
- Arborele cel mai simplu: combină perechi de produse parțiale $\rightarrow$ numărul produselor parțiale ar fi redus de la n la $n/2$

8. Circuit de înmulțire matriceală pipeline utilizând celula de 1 bit de înmulțire și adunare

- Considerăm înmulțirea a două numere întregi de n biți $X = x_{n-1} x_{n-2} \dots x_0$, $Y = y_{n-1} y_{n-2} \dots y_0$
- Circuitele de înmulțire matriceală pot fi modificate în mod simplu pentru a utiliza tehnica pipeline Exemplu: Circuit de înmulțire matriceală pipeline care utilizează celula M de 1 bit de înmulțire și adunare Cele n celule din fiecare etaj E_i calculează un produs parțial $P_i = P_{i-1} + X * 2^i$



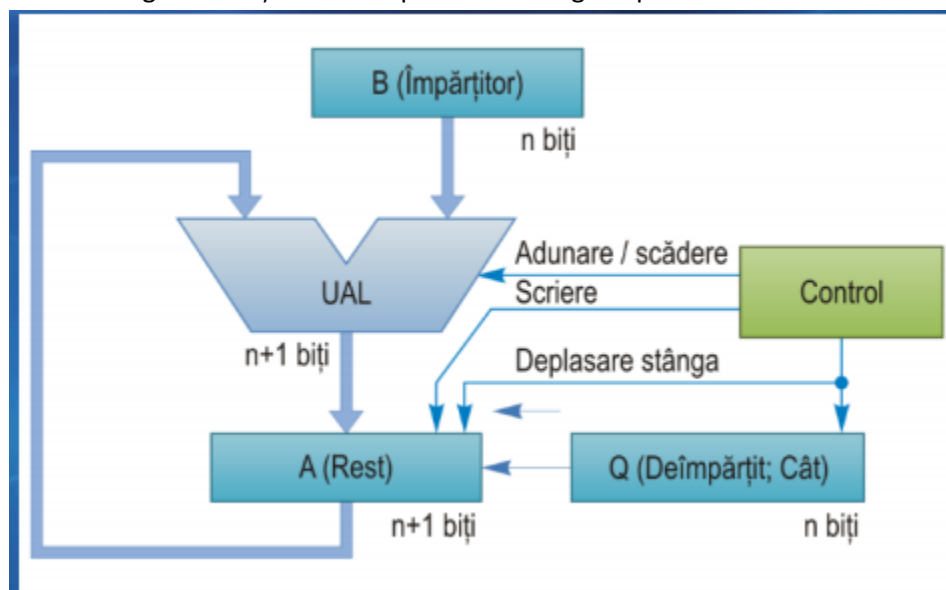
9. Circuit de înmulțire pipeline utilizând un arbore Wallace pentru însumarea produselor parțiale



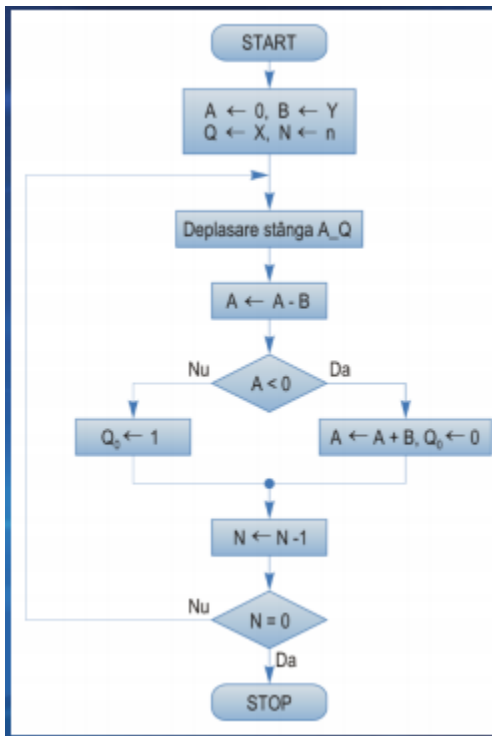
Curs 6 Impartire

1. Versiunea finală a circuitului de împărțire cu refacerea restului parțial

- Deplasarea restului parțial la stânga în locul deplasării împărțitorului la dreapta:
 - Produce aceeași aliniere
 - Simplifică circuitele necesare pentru UAL și registrul împărțitorului (n biți în loc de $2n$)
- A doua îmbunătățire: primul pas nu poate genera o cifră de 1 în cadrul câtului
 - Inversarea ordinii operațiilor: deplasare, apoi scădere → se poate elimina o iterație
- Dimensiunea registrului A poate fi redusă la jumătate
- Registrele A și Q pot fi combinate
 - Se deplasează biții deîmpărțitului în registrul A în loc de a deplasa zerouri
 - Registrele A și Q sunt deplasate la stânga împreună



2. Versiunea finală a algoritmului de împărțire cu refacerea restului parțial



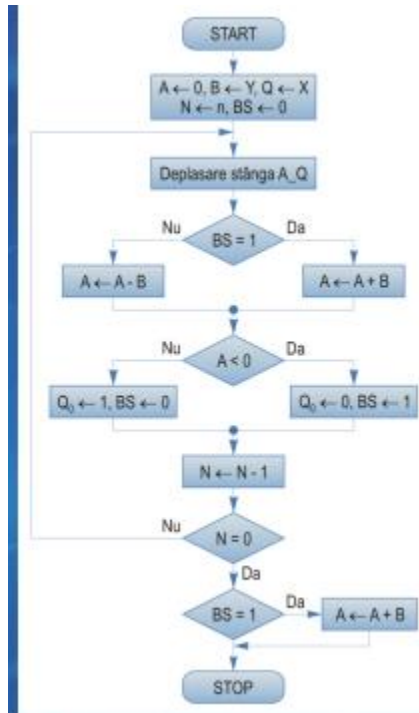
3. Principiul metodei de împărțire fără refacerea restului parțial

- După deplasarea la stânga a restului parțial, operația depinde de rezultatul din pasul precedent
- Restul parțial a fost pozitiv: scădere
- Restul parțial a fost negativ: adunare

4. Algoritmul de împărțire fără refacerea restului parțial

- Se deplasează registrele A_Q la stânga
- Dacă în pasul precedent restul parțial a fost pozitiv, se scade împărțitorul din restul parțial
- Dacă restul parțial a fost negativ, se adună împărțitorul la restul parțial

- După ultimul pas, dacă restul parțial este negativ, restul trebuie refăcut



- Se utilizează un bistabil suplimentar BS
- BS indică operația care trebuie efectuată:
 - BS = 0: scădere
 - BS = 1: adunare
- În prima etapă se efectuează o scădere → BS este inițializat cu 0

5. Principiul metodei de împărțire cu refacerea restului parțial

- În fiecare etapă, se efectuează deplasarea la stânga a restului parțial și scăderea împărțitorului din restul parțial
- Dacă se obține un rezultat negativ, restul parțial trebuie refăcut

● Împărțirea cu refacerea restului parțial:

- $R \leftarrow R - Y$
- $R \leftarrow R - Y + Y$
- $R \leftarrow 2 \times R$
- $R \leftarrow 2 \times R - Y$

● Împărțirea fără refacerea restului parțial:

- $R \leftarrow R - Y$
- $R \leftarrow 2 \times R - 2 \times Y$
- $R \leftarrow 2 \times R - 2 \times Y + Y$

Curs 7

Virgula mobila

- Reprezentarea în virgulă mobilă (VM) a numărului N :
 - $N = (-1)^s \times b \times e \times m$
 - s – semn (0 sau 1)
 - b – bază
 - e – exponent: ordinul de mărime al numărului
 - m – mantisă: valoarea exactă a numărului într-un anumit domeniu

1. Reprezentare în VM cu exponent deplasat

- De obicei, nu se reprezintă exponentul real → exponent deplasat (caracteristică)
 $E = e + \text{deplasament}$ (Exponentul deplasat este întotdeauna pozitiv)
- Avantaje ale exponentului deplasat:
 - Simplificarea operațiilor cu exponentul
 - Reprezentarea numărului zero
 - Mantisă: cifre de 0 în toate pozițiile
 - Exponentul: poate avea, teoretic, orice valoare → “zero impur” sau “zero pur”
- Dezavantaj al exponenților deplasați:
 - Adunarea exponenților este mai complicată → deplasamentul trebuie scăzut din suma exponenților

2. Reprezentare normalizată în VM

- Bitul c.m.s. al mantisei este 1
- Avantaje: operațiile sunt simplificate și precizia este crescută
- Deoarece bitul c.m.s. este 1, acest bit nu este memorat → bit ascuns
- În unele cazuri, bitul ascuns se presupune poziționat la stânga virgulei binare → 1,m
- Depășire superioară: exponentul depășește valoarea maximă (ex., $e > 127$)
- Depășire inferioară: exponentul are o valoare negativă prea mică (ex., $e < -128$)

3. Formate specificate de standardul IEEE 754 pentru numere în VM

- Formate pentru operații aritmetice cu date binare și zecimale în VM
- Formate pentru interschimbarea datelor
- Operații de bază în VM
- Conversii între: diferite formate în VM; formate întregi și în VM; formate în VM și reprezentări externe (șiruri de caractere)
- Reguli de rotunjire

4. Valori speciale specificate de standardul IEEE 754 pentru numere în VM

- Standardul permite reprezentarea unor valori speciale
- Valoarea zero
 - $E = 0$, Frație = 0
 - Două reprezentări pentru valoarea 0: +0, -0
 - Bitul ascuns de la stânga virgulei binare este implicit 0 în loc de 1
- Numere nenormalizate
 - $E = 0$; Frație $\neq 0$; bitul ascuns este 0
 - Un număr nenormalizat este generat prin tehnica numită depășire inferioară graduală
 - Format cu precizie simplă → exponentul real minim este -126
 - Rezultatul necesită un exponent egal cu -129 pentru a se obține un număr normalizat
- Valoarea infinit
 - E = valoarea maximă pentru formatul utilizat, Frație = 0
 - Două reprezentări pentru infinit: $+\infty$, $-\infty$
 - Valoarea infinit se poate utiliza ca operand: $\infty + n = \infty$; $n / \infty = 0$; $n / 0 = \infty$
- NaN (Not a Number)
 - S-a prevăzut pentru indicarea unor excepții
 - Operații nedefinite: ∞/∞ , $\infty - \infty$, $\infty * 0$, $0/\infty$, $0/0$
 - E = valoarea maximă, Frație $\neq 0$
 - Două tipuri de valori NaN:
 - qNaN: nu se semnalează o excepție ("quiet" NaN)
 - sNaN: este semnalată o excepție ("signaling" NaN)
- Formate zecimale
 - Prevăzute pentru aplicații comerciale
 - Un număr poate avea reprezentări multiple → sunt admise zerouri în pozițiile c.m.s. ale mantisei
 - Cohortă: set de reprezentări ale unui număr

5. Excepții specificate de standardul IEEE 754 pentru numere în VM

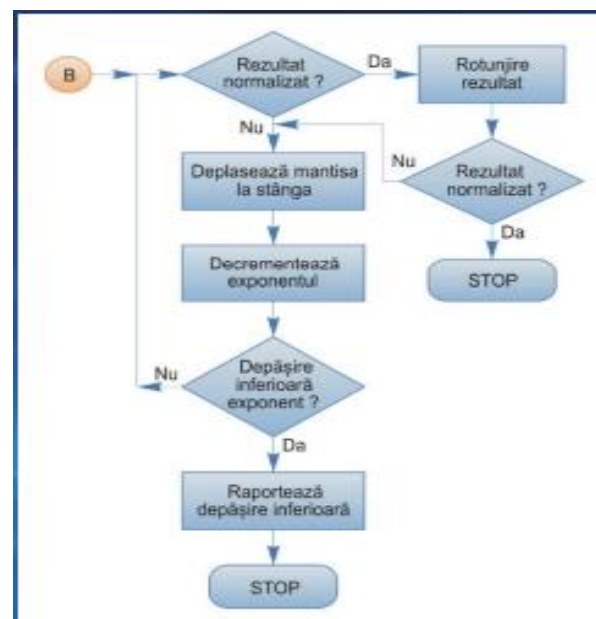
- Depășire inferioară
- Depășire superioară
- Împărțire la zero
- Rezultat inexact: rezultatul trebuie rotunjit
- Operație invalidă: apare în cazul unor operații ca $0/0$ sau $\infty - \infty$
- Implicit, la apariția unei excepții este setat un indicator și calculele continuă

6. Avantaje/dezavantaje ale standardului IEEE 754

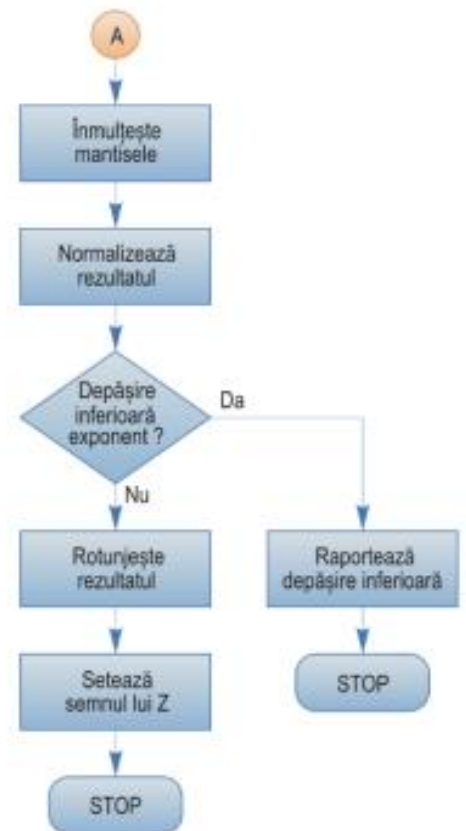
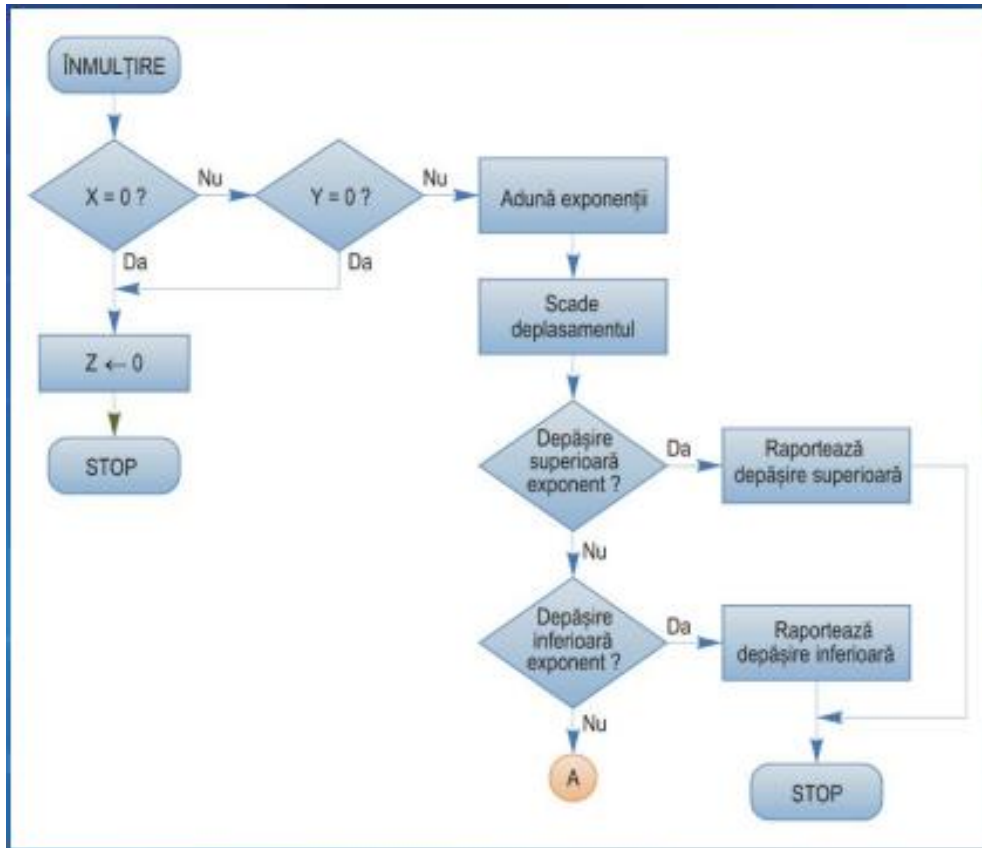
- Avantaje ale standardului IEEE 754:
 - Facilitează scrierea unor programe portabile
 - Definește condiții pentru scrierea programelor reproductibile → produc același rezultat cu diferite implementări ale unui limbaj
- Dezavantaje
 - Conține specificații opționale
 - Implementare lentă a depășirii inferioare graduale comparativ cu setarea la zero
 - Nu se specifică aritmetica pt. valori complexe

7. Algoritmul de adunare/scădere în VM

- Pentru adunare sau scădere, trebuie să se realizeze egalizarea exponenților numerelor
 - Compararea mărimii exponenților
 - Alinierea mantisei numărului cu exponentul mai mic
- Etapele algoritmului
 - Alinierea mantiselor
 - Adunarea sau scăderea mantiselor
 - Normalizarea rezultatului
 - Rotunjirea rezultatului



8. Algoritmul de înmulțire/împărțire în VM



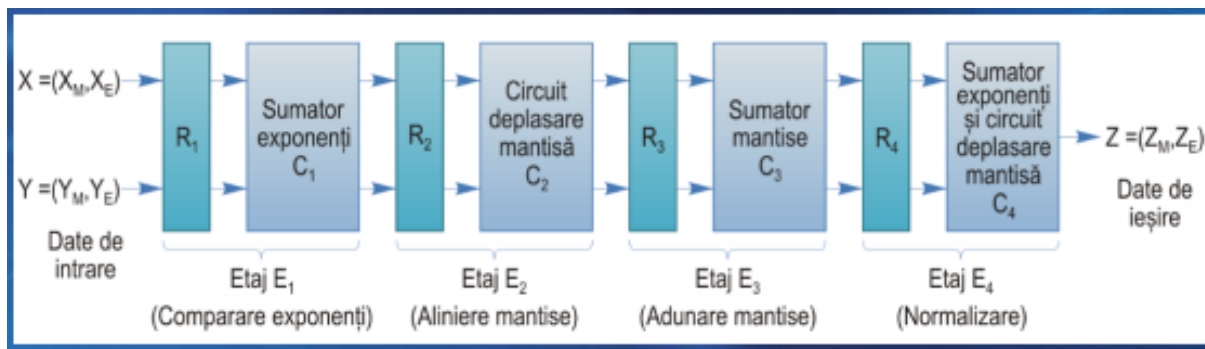
9. Cifre de gardă și de rotunjire

- Cifre de gardă g :
 - Sunt plasate la dreapta primelor p cifre ale mantisei pentru a evita pierderea cifrelor
- Cifre de rotunjire r :
 - Dacă rezultatul este normalizat, dar există cifre diferite de zero la dreapta mantisei, numărul trebuie rotunjit
 - Sunt plasate la dreapta cifrelor de gardă
 - După deplasarea la stânga a rezultatului pentru normalizare, acesta poate fi rotunjit

10. Moduri de rotunjire specificate de standardul IEEE 754 pentru numere în VM

- Rotunjire spre plus infinit (rotunjire în sus)
- Rotunjire spre minus infinit (rotunjire în jos)
- Rotunjire spre zero (trunchiere)
- Rotunjire la cel mai apropiat număr par
- Ultimul mod: prevăzut pentru situațiile în care numărul real se află exact la jumătatea intervalului dintre două reprezentări în VM Dacă bitul c.m.p.s. este impar, se adună 1 Dacă bitul este par, numărul este trunchiat Pentru a se obține o rotunjire corectă în ultimul caz, este necesar un alt tip de cifră suplimentară → bit de colectare s (sticky bit)
- Pentru operațiile zecimale este specificat un mod de rotunjire suplimentar
- Rotunjire la cel mai apropiat număr, cu îndepărtare de la zero –
 - Dacă numărul real se află exact la jumătatea intervalului dintre două reprezentări în VM, este rotunjit la valoarea cea mai apropiată cu mărimea mai mare

11.Schema bloc a unui sumator pipeline în VM



- Etape de proiectare pentru un circuit aritmetic pipeline
 - Găsirea unui algoritm secvențial cu mai multe etape → etapele trebuie să fie echilibrate
 - Plasarea unor registre buffer între etaje

